IN-MEMORY KEY-VALUE STORE (Redis-lite)

Phase 1 — Basic TCP Server (Echo). C++20, POSIX Sockets | What we built, line by line.

1 — WHAT WE ARE BUILDING

The goal

A program that stores data as **key → value** and is reachable **over the network**, so other programs (clients) can connect and run commands.

```
"name" -> "shivam"
"city" -> "surat"
```

Internally it is just an `unordered_map` — exposed over a network.

Phase 1 scope (today)

- Server listens on port **6379**
- One client connects
- Whatever client sends → sent back (**echo**)

No storage. No commands. Only connection + data flow.
Echo first — to prove the connection works before adding GET/SET.

> **REAL-LIFE — CLOAKROOM**
> You hand over a bag → get a **TOKEN (#42)**. Show the token → get the bag back. **TOKEN = KEY**, **BAG = VALUE**, counter = the KV store.

2 — SOCKET = A PHONE CALL BETWEEN TWO PROGRAMS

Your `unordered_map` lives *inside* one program. A second program cannot reach inside it. A **socket** is the connection (the "phone line") that lets them talk.

Server = the shop's landline

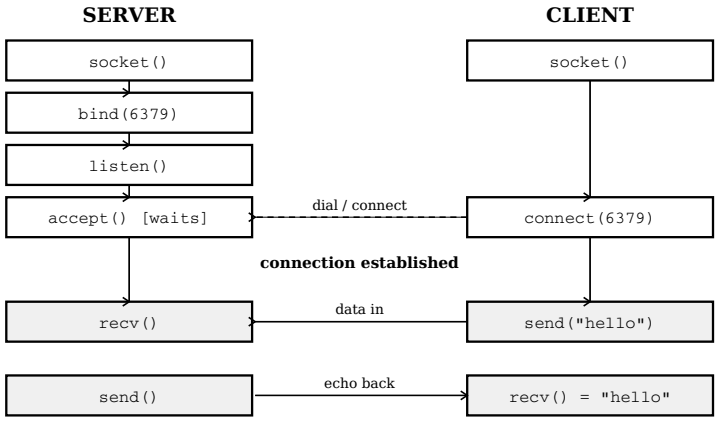
Always ON, waits for calls. Does the 5 steps below.

Client = the caller

Dials the number, then talks. Only 3 steps.

> **FILE DESCRIPTOR (FD)**
> In Linux everything is a **NUMBER**. Open a socket → you get a number (the fd). That number is the "handle" you use to listen / send / close. **FD = THE TOKEN THAT LETS YOU ACCESS THE SOCKET.**

3 — THE FLOW (server 5 steps, client 3 steps)



Step	Server	Meaning (real-life)
`socket()`	make endpoint	buy a phone
`bind()`	assign IP + port 6379	give the phone a number
`listen()`	ready for calls	switch phone ON, let it ring
`accept()`	take one connection	pick up the call (waits until one comes)
`recv()`/`send()`	read / write data	listen / speak

4 — THE CODE, LINE BY LINE

Headers

Line	What it gives
`#include <iostream>`	cout / cerr — printing
`#include <cstring>`	memset — clear a buffer
`#include <sys/socket.h>`	socket, bind, listen, accept, recv, send (POSIX)
`#include <netinet/in.h>`	sockaddr_in — the address struct (IP + port)
`#include <unistd.h>`	close() — release an fd

These are **POSIX** headers (Linux/Unix). This is what makes it real systems code — not Windows Winsock.

1. Create the socket

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
if (server_fd == -1) { std::cerr << "socket fail\n"; return 1; }
```

Part	Meaning
`AF_INET`	use IPv4 (which "world" of addresses)
`SOCK_STREAM`	TCP — reliable, ordered (SOCK_DGRAM = UDP)
`0`	protocol = default (picks TCP automatically)
`== -1`	functions return -1 on failure — always check (error handling)
`server_fd`	the socket's fd (its handle/token)

ANALOGY Bought a phone. `SOCK_STREAM` = it's a TCP phone (your reliable kind). `server_fd` = the phone's handle.

2. Fill the address box

```
sockaddr_in address;
memset(&address, 0, sizeof(address));
address.sin_family      = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port        = htons(6379);
```

Line	Meaning
`sockaddr_in address;`	a struct (box) holding family + IP + port
`memset(...0...)`	wipe the box clean — a fresh struct has garbage; clear before filling
`sin_family = AF_INET`	IPv4 — same as in socket(), stay consistent
`s_addr = INADDR_ANY`	listen on ANY of this machine's IPs
`sin_port = htons(6379)`	port 6379, converted to network byte order

HTONS() — WHY?
CPU and network store numbers in different byte order. `htons` = "Host TO Network Short" = translate the port so the other machine reads it correctly. **ALWAYS WRAP THE PORT IN HTONS.**
MEMSET ANALOGY: clear/blank a new form fully before writing your own values on it.

3. Bind — attach the address to the socket

```
if (bind(server_fd, (sockaddr*)&address, sizeof(address)) == -1) {
    std::cerr << "bind fail — port busy?\n"; return 1;
}
```

Arg	Meaning
`server_fd`	which socket gets the address
`(sockaddr*)&address`	the address box, cast to the generic type bind expects
`sizeof(address)`	how big that box is

Fails (`-1`) if the port is already in use by another program. The cast `(sockaddr*)` is a required formality — bind takes a generic `sockaddr`.

ANALOGY The phone now has its number (6379). Built, numbered — but still OFF.

4. Listen + Accept

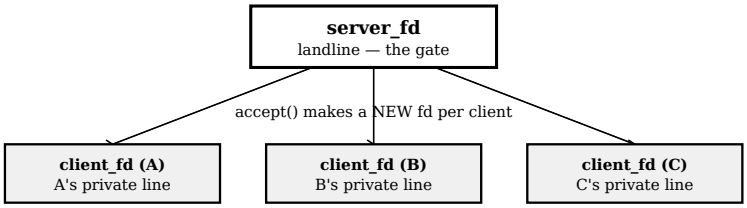
```
listen(server_fd, 5);                 // turn ON, allow 5 waiting
```

```
int client_fd = accept(server_fd, nullptr, nullptr);  // pick up
```

Part	Meaning
`listen(fd, 5)`	socket goes live; "5" = backlog (5 callers can wait in line)
`accept(server_fd...)`	blocks (waits) until a client connects; returns a NEW fd
`client_fd`	that new fd — the private line to THIS client
`nullptr, nullptr`	"don't need the client's address right now"

ANALOGY `listen` = phone ON, ringing allowed; backlog = waiting line. `accept` = pick up the call (you wait here until someone calls).

KEY IDEA — server_fd vs client_fd



recv() / send() happen on the private lines — NOT on server_fd

one server_fd · a separate client_fd per connection

PIZZA SHOP The landline (server_fd) only connects calls. Each customer gets a private line (client_fd) where the real order happens. The landline stays free for the next caller.

5. Echo loop — recv then send back

```
char buffer[1024];
while (true) {
    memset(buffer, 0, sizeof(buffer));
    int bytes = recv(client_fd, buffer, sizeof(buffer), 0);
    if (bytes <= 0) { break; }            // client left
    send(client_fd, buffer, bytes, 0);    // echo back
}
```

Line	Meaning
`char buffer[1024]`	a 1024-byte box to hold incoming data (a notepad)
`while (true)`	client may send many messages — keep listening
`memset(buffer,0,..)`	clear old data before each read
`recv(client_fd,...)`	read from the PRIVATE line into buffer; returns bytes read; blocks until data
`bytes <= 0`	0 = client closed, <0 = error → break the loop
`send(client_fd, buffer, bytes, 0)`	send back exactly "bytes" (only what arrived, not full 1024)

WHY `BYTES`, NOT `SIZEOF(BUFFER)`? Client sent 5 bytes ("hello"). Send back 5 — not 5 real + 1019 garbage. Echo = exactly what came in.

6. Cleanup

```
close(client_fd);   // close the client line
close(server_fd);   // close the landline
return 0;
```

`close()` frees the fd. Done with work → close both, so resources don't leak.

5 — COMPILE & RUN (WSL / Linux)

Terminal 1 — server

```
g++ -std=c++20 main.cpp -o server
./server
```

Output:

```
Server ready on port 6379...
```

Cursor pauses here = correct. It is stuck at `accept()`, waiting for a client.

Terminal 2 — client

```
nc localhost 6379
hello shivam        <- you type
hello shivam        <- server echoes
```

Server side prints `Client connected!`
(If nc missing: `sudo apt install netcat-openbsd -y`)

Why "hello" shows twice: first line = your typing; second = the server's echo. **Stop:** Ctrl+C in each terminal.

6 — DONE / ROADMAP

Phase 1 — verified working

- socket → bind → listen → accept
- recv → send (echo)
- server_fd vs client_fd understood
- compiled with g++, tested with nc

Next phases

- **2 — Commands:** parse GET / SET / DEL
- 3 — Storage + thread safety (hashmap + sharded locks)
- 4 — TTL (auto-expiry)
- 5 — LRU eviction
- 6 — Benchmark (50K+ ops/sec)